

LIST OF FIGURES

Figure 1: FocusFlow Application Architecture Diagram1

Figure 2: Zustand State Management Flow2

Figure 3: Recovery Mode Activation Logic3

Figure 4: Dashboard Module — Grid Layout Structure4

Figure 5: Conversational Reset — Sentiment Analysis Flow5

Figure 6: Focus Timer — Circular SVG Progress Arc6

Figure 7: Habit Card — Weekly Progress Component7

Figure 8: Habit Heatmap — Calendar Visualisation8

Figure 9: Note Canvas — Block Editor Architecture9

Figure 10: Analytics Module — Habit Strength Radar Chart10

Figure 11: Sidebar — macOS-Style Dock Wave Effect.....11

Figure 12: Dark Mode Theming via CSS Custom Properties.....12

LIST OF TABLES

Table 1: Technology Stack Summary1

Table 2: Zustand Store Inventory2

Table 3: Navigation Routes and Module Mapping3

Table 4: Note Block Types and Descriptions4

Table 5: Analytics Computation Functions5

Table 6: Recovery Mode Behavioural Changes6

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ADHD	Attention Deficit Hyperactivity Disorder
API	Application Programming Interface
CSS	Cascading Style Sheets

Abbreviation	Full Form
CSR	Client-Side Rendering
DXA	Device-independent Pixel (twips unit)
EQ	Emotional Quotient / Emotional Intelligence
HTML	Hypertext Markup Language
JSON	JavaScript Object Notation
JSX	JavaScript XML (React syntax extension)
MVC	Model–View–Controller
PWA	Progressive Web Application
SPA	Single Page Application
SSR	Server-Side Rendering
TSX	TypeScript XML (TypeScript React syntax)
UI	User Interface
UX	User Experience

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

The global productivity software market is among the fastest-growing segments of the enterprise technology sector. Tools such as Notion, Todoist, Trello, ClickUp, and Microsoft To Do have attracted hundreds of millions of users, collectively facilitating the management of tasks, projects, and habits at both individual and organisational scales. However, a significant and underserved gap persists in this landscape: almost all mainstream productivity applications are designed for optimal, high-functioning states of mind. They implicitly assume that the user is alert, motivated, and cognitively available at all times.

This assumption fails a substantial proportion of the global workforce. Research on burnout, mental health, neurodivergent conditions such as Attention Deficit Hyperactivity Disorder (ADHD), and the growing recognition of cyclical energy patterns indicates that productivity tools must evolve beyond simple task lists and completion percentages. An application that punishes missed streaks, presents an overwhelming volume of information at low-energy moments, or surfaces aggressive metrics can actively worsen the psychological state of a user who is already struggling.

FocusFlow was conceived as a direct response to this gap. The project aims to build a production-grade productivity web application grounded in a Recovery-First, Emotionally Intelligent (EQ-first) philosophy. Rather than demanding performance, FocusFlow meets users where they are — responding differently to users who are overwhelmed, rested, or energised — and gently guides them toward sustainable engagement with their work.

1.2 Problem Statement

Existing productivity applications exhibit the following systemic deficiencies that FocusFlow seeks to address.

First, conventional tools provide a static interface regardless of the user's energy state or recent activity patterns. A user returning after several unproductive days is greeted with the same dense dashboard and task backlog as a user operating at peak capacity, creating a re-entry barrier that can lead to avoidance and disengagement.

Second, the predominant feedback mechanisms in productivity applications are punitive rather than supportive. Broken streaks, red overdue indicators, and completion percentage drops contribute to shame spirals that demotivate users rather than re-engaging them.

Third, most productivity tools lack any form of emotional or contextual awareness. They do not ask how the user is feeling, they do not adapt their complexity to the user's current capacity, and they do not distinguish between a user who is "done for the day" and one who is genuinely in distress.

FocusFlow addresses each of these gaps through its adaptive Recovery Mode, the avoidance of negative feedback colours and imagery, and the Conversational Reset widget, which introduces rudimentary emotional sensing into the dashboard experience.

1.3 Project Objectives

The primary objectives of the FocusFlow project are as follows.

- To design and implement a multi-module productivity web application using a modern React-based technology stack.
- To introduce a Recovery Mode system that automatically reduces interface complexity based on user inactivity, improving re-entry experience for users returning after unproductive periods.
- To implement an emotionally intelligent Conversational Reset widget capable of detecting sentiment from free-text input and generating contextually appropriate productivity suggestions.
- To build a circular, SVG-based Focus Timer with Pomodoro-style interval support, session logging, and motivational feedback upon completion.
- To develop a Habit Tracker featuring weekly consistency scoring, a heatmap calendar, hydration tracking, and a radar chart visualisation of habit strength.
- To create a flexible, block-based note editor supporting multiple content types including text, headings, checklists, quotations, dividers, and voice memos.
- To design a passive Analytics module that presents user data through gentle narrative insights, avoiding comparative, punitive, or anxiety-inducing metrics.
- To deliver a fully responsive interface that functions consistently across desktop and mobile devices.

1.4 Scope and Delimitations

FocusFlow is scoped as a client-side, single-user application for the purposes of this project. State persistence is achieved through Zustand's persist middleware, which serialises store data to localStorage. The application does not include a backend server, REST API, or database in its current phase. User authentication, multi-device synchronisation, and cloud storage are explicitly deferred to a future development phase.

The application is designed for deployment as a Next.js web application. While the responsive design supports mobile browsers, it is not packaged as a native mobile application or Progressive Web App (PWA) in Phase 4. Accessibility optimisations beyond standard semantic HTML and focus-visible outlines are also identified as future work.

1.5 Report Organisation

The remainder of this report is structured as follows. Chapter 2 presents a review of relevant literature and prior work on productivity tools, emotional intelligence in software, and neurodiversity-inclusive design. Chapter 3 describes the overall system architecture, technology stack, and design system. Chapter 4 details the methodology and implementation of each application module. Chapter 5 presents experimental results and a discussion of the application's capabilities. Chapter 6 offers conclusions and outlines directions for future work.

CHAPTER 2

LITERATURE STUDY

2.1 Productivity Tools: Current Landscape

The evolution of personal productivity software began with simple to-do lists in the early 2000s, exemplified by applications such as Remember the Milk (2004) and Toodledo. The subsequent decade saw a shift toward kanban-style visual project management with Trello (2011), followed by the rise of all-in-one knowledge management systems such as Notion (2016) and Coda. Today, the market is characterised by feature-rich platforms offering task management, calendar integration, document creation, and team collaboration within unified workspaces.

Despite their functional richness, these tools share a fundamental design paradigm: they measure user value through completion and output. Research by Cirillo (2006) on time management and by Newport (2016) on the mechanics of deep work established that focused, uninterrupted work blocks produce disproportionate cognitive output relative to fragmented, multitasking work sessions. These findings informed the emergence of Pomodoro-based timer applications, which FocusFlow’s Deep Work module directly implements.

A persistent criticism of productivity software, articulated by scholars of human-computer interaction, is the “calendar colonisation” effect — the tendency of task management tools to create an always-on obligation that itself becomes a source of cognitive load. FocusFlow’s Simplify Mode and Recovery Mode represent a practical software-engineering response to this critique.

2.2 Emotional Intelligence in Software

The concept of Emotional Intelligence (EQ), introduced by Salovey and Mayer (1990) and popularised by Goleman (1995), describes the capacity to perceive, understand, manage, and apply emotional information. Its application to software design — often termed “affective computing” — was pioneered by Picard (1997), who proposed that computers should be capable of recognising and responding to human emotional states.

In the domain of productivity software, EQ manifests as contextual awareness — the ability of an application to infer user state from available signals and adapt its behaviour accordingly. FocusFlow implements a simplified, rule-based form of affective computing through two mechanisms. The first is the Recovery Mode system, which infers diminished user capacity from temporal inactivity data. The second is the Conversational Reset widget, which applies keyword-based sentiment classification to free-text user input, assigning one of three states (low, neutral, or high energy) and generating targeted suggestions.

While FocusFlow’s sentiment engine is deterministic rather than machine-learned, it establishes an architectural foundation that could readily incorporate a natural language processing (NLP) model in a future iteration, extending affective awareness to nuanced linguistic cues beyond keyword matching.

2.3 Neurodiversity-Inclusive Design

Approximately 15–20% of the global population is estimated to be neurodivergent, a term encompassing ADHD, dyslexia, autism spectrum conditions, and related differences in cognitive processing. Research by Doyle (2020) and advocacy organisations such as the ADHD Foundation consistently highlight the inadequacy of standard productivity tools for neurodivergent users, who are disproportionately affected by overwhelming interfaces, punitive feedback, and rigid time management frameworks.

FocusFlow’s design makes several neurodiversity-inclusive choices. The deliberate exclusion of red as a feedback colour — noted in the `globals.css` design token definitions, where `--feedback-success` maps to cyan rather than green and no red token exists — reduces anxiety-triggering visual signals. The Recovery Mode’s task limitation to three items per view aligns with research on working memory constraints in ADHD, which suggest that presenting fewer than four items at a time improves task initiation. The Conversational Reset widget’s explicitly non-judgmental framing (“No judgment here”) and its “Not Yet” option further reduce the pressure of re-engagement.

2.4 Deep Work and the Pomodoro Technique

Cal Newport’s concept of “deep work” (2016) refers to professional activities performed in a state of distraction-free concentration that push cognitive capabilities to their limit. Newport argues that deep work is both increasingly rare and increasingly valuable in the knowledge economy. The Pomodoro Technique, developed by Francesco Cirillo in the late 1980s, operationalises focused work through a timer-based protocol: 25 minutes of focused work followed by a 5-minute break, with longer breaks every four cycles.

FocusFlow’s Deep Work module operationalises these principles through a circular SVG timer offering preset durations of 15, 25, 45, and 60 minutes. The module logs completed sessions with their start timestamps and durations, enabling the Analytics module to compute daily and weekly focus patterns. A noise selector component is also provided, acknowledging research findings on the concentration-enhancing effects of ambient noise at moderate volumes (approximately 70 dB).

2.5 Habit Formation Frameworks

The scientific basis for habit tracking in productivity software draws from Lally et al.’s (2010) research on habit formation timelines, which found that new behaviours take an average of 66 days to become automatic. James Clear’s “Atomic Habits” framework (2018) further argues for the primacy of systems over goals, and for identity-based habit formation over outcome-based motivation.

FocusFlow’s Habit Tracker implements these principles through weekly frequency targets rather than daily streaks, reducing the penalty of missed days and supporting flexible scheduling. The HabitStrengthCard component computes a weekly consistency score as a percentage, and the HabitHeatmap provides a calendar-based visualisation of historical completion patterns. The deliberate avoidance of a streak counter — a standard feature in applications like Habitica and Streaks — reflects the design philosophy of deprioritising performance anxiety.

2.6 Technology Review

The following technologies were evaluated during the project planning phase.

Next.js 14 with the App Router was selected as the primary framework due to its support for React Server Components, file-based routing, built-in image optimisation, and its status as the de facto standard for production React applications as of 2024. The App Router’s layout system enables the persistent AppShell component — containing the sidebar and top navigation — to wrap all page routes without remounting on navigation.

Zustand was chosen over Redux and React Context for state management due to its minimal boilerplate, excellent TypeScript support, and first-class persistence via the persist middleware. Each application domain (tasks, habits, notes, deep work, UI, user) is represented as an independent Zustand store, enabling clean separation of concerns.

Framer Motion was selected for animations due to its declarative API, layout animation support, and robust AnimatePresence component for handling exit animations — a requirement for the dashboard’s Conversational Reset, which transitions between input, response, and “not yet” states.

Tailwind CSS was chosen for styling due to its utility-first approach, which aligns well with component-level styling in React, and its dark-mode class strategy that integrates cleanly with the next-themes library.

CHAPTER 3

SYSTEM ARCHITECTURE AND DESIGN

3.1 Technology Stack

Table 1 summarises the complete technology stack employed in FocusFlow.

Layer	Technology	Version / Notes
Framework	Next.js (App Router)	v14.x — file-based routing, React Server Components
UI Library	React	v18.x — Hooks, Suspense, concurrent rendering
Language	TypeScript	v5.x — strict mode, typed stores and props
Styling	Tailwind CSS	v3.x + CSS custom properties (design tokens)
State Management	Zustand	v4.x — with persist middleware for localStorage
Animations	Framer Motion	v11.x — layout, AnimatePresence, spring physics
Icons	Lucide React	v0.383.0 — consistent stroke-width icon set
Theming	next-themes	Dark/light mode via class strategy
Fonts	Google Fonts (Inter, Lora)	Loaded via next/font for performance
Deployment Target	Vercel / Local Next.js Server	Phase 4 — LOCAL

3.2 Application Architecture

FocusFlow follows a layered architecture comprising four principal layers: routing, shell, content, and state.

The routing layer is implemented via Next.js App Router. Each directory within the /app folder corresponds to a navigable route. Server-side page.tsx files serve as entry points and import client-side Content components, maintaining the boundary between React Server Components and client-side interactive components. The application's root page.tsx performs an immediate server-side redirect to /dashboard, establishing the dashboard as the default landing page.

The shell layer is implemented by the AppShell component, which wraps all page content through the /app/layout.tsx root layout. AppShell renders the persistent Sidebar, TopBar, SimplifyModeBanner, and ToastContainer. The Sidebar renders in desktop mode as a collapsible vertical panel and in mobile mode as a bottom navigation bar with five primary links. This responsive shell pattern ensures consistent navigation across all viewport sizes.

The content layer consists of the six Content components — DashboardContent, TasksContent, DeepWorkContent, HabitsContent, NotesContent, and AnalyticsContent — each marked with the 'use client' directive, enabling full access to React hooks, browser APIs, and Zustand stores.

The state layer is composed of six independent Zustand stores, each responsible for a distinct application domain. Cross-cutting state such as the recovery mode determination is encapsulated in a custom hook (useRecoveryMode) that reads from multiple stores to produce a single boolean value.

3.3 State Management Strategy

Table 2 provides an inventory of the Zustand stores and their primary responsibilities.

Store	Managed State	Key Actions
taskStore	Task array (id, title, completed, completedAt, timerMinutes)	addTask, completeTask, deleteTask
habitStore	Habit definitions, log entries, strength score computation	addHabit, logHabit, deleteHabit, isTodayLogged, getStrengthScore
deepWorkStore	Session state (active, paused, timer), session history array	startSession, pauseSession, stopSession, setDuration
noteStore	Note array with block-level content, active note ID	createNote, addBlock, updateBlockContent, reorderBlocks

Store	Managed State	Key Actions
uiStore	Sidebar collapse, simplify mode, analytics reveal state	setSidebarCollapsed, setSimplifyMode, setAnalyticsRevealed
userStore	Energy state, lastProductiveAt, lastActiveAt timestamps	setEnergyState, setLastProductiveAt, setLastActiveAt

All stores are initialised with persist middleware, which serialises the complete store state to localStorage under a namespaced key on every state mutation. This approach eliminates the need for an API server for the current development phase while providing seamless state restoration across browser sessions.

3.4 Design System and Theming

FocusFlow implements a comprehensive design token system via CSS custom properties defined in globals.css. The token vocabulary encompasses brand colours (--brand-blue, --brand-cyan, --brand-teal, --brand-navy), semantic background levels (--bg-base, --bg-card, --bg-sidebar, --bg-muted), text hierarchy tokens (--text-primary, --text-secondary, --text-muted), spacing scale (--space-1 through --space-12), border radius scale (--radius-sm through --radius-full), and shadow definitions (--shadow-xs through --shadow-fab).

Dark mode is implemented through a .dark selector block in globals.css that overrides the custom property values, following the next-themes class strategy. This approach ensures that all styled components automatically respond to theme changes without conditional class application. The absence of red from the feedback colour palette is an intentional design decision: --feedback-success maps to cyan (#22D3EE) and --feedback-warning maps to amber (#C9A96E), avoiding colours that carry negative psychological associations in Western cultural contexts.

Reusable visual patterns — the glass card (.glass-card), gradient text (.gradient-text), gradient background (.gradient-bg), and recovery mode overlay (.recovery-mode-bg) — are defined as Tailwind component classes, enabling consistent application across components without repeated inline styling.

3.5 Recovery Mode Architecture

Recovery Mode is the defining architectural feature of FocusFlow and warrants detailed treatment. The mode is governed by the useRecoveryMode custom hook, which evaluates whether the difference between the current timestamp and the lastProductiveAt value stored in userStore

exceeds a configurable threshold (the default implementation targets a 24-hour window but can be parameterised).

When Recovery Mode is active, the following behavioural changes are applied across the application. The AppShell forces the sidebar into its collapsed (icon-only) state by calling `setSidebarCollapsed(true)` in a `useEffect` hook. The background transitions to the `.recovery-mode-bg` soft gradient overlay. The DashboardContent limits the displayed task list to a maximum of three items and changes the heading from “Today’s focus” to “Start with one small thing.” The RecoveryBanner component is rendered at the top of the dashboard, displaying the number of days since the last productive session. The Gentle Reminders sidebar widget is hidden. The AnalyticsContent replaces the full analytics panel with a compassionate message indicating that the numbers can wait.

The design intent of Recovery Mode is to lower the activation energy required to re-engage with the application after a period of non-use, reducing the perceived cost of return and providing a low-stakes entry point back into productive behaviour.

CHAPTER 4

METHODOLOGY AND MODULE IMPLEMENTATION

4.1 Development Methodology

FocusFlow was developed using an incremental, phase-based methodology across four development phases. Phase 1 established the project scaffold, routing structure, and global design system. Phase 2 implemented the core productivity modules (Tasks, Deep Work, Habits). Phase 3 introduced the Notes module and Analytics infrastructure. Phase 4 integrated the Recovery Mode system, Conversational Reset, and cross-module polish.

Each phase followed a component-first development pattern: data models and Zustand stores were defined first, followed by leaf components (e.g., HabitCard, TaskCard, FocusTimer), then page-level Content components that assembled the leaf components, and finally route-level page wrappers. This bottom-up approach facilitated isolated component testing and prevented dependency inversions.

TypeScript strict mode was enforced throughout, with all component props, store types, and computed values explicitly typed. The use of TypeScript's discriminated union types was particularly valuable in the Notes module, where the NoteBlock type is a union of BlockType variants, each with different associated data fields.

4.2 Dashboard Module

The Dashboard serves as the application's primary landing page and emotional entry point. Its layout follows a responsive three-column grid on large screens, collapsing to a single column on mobile. The left two-thirds contain the Conversational Reset widget, a Task Snapshot card, and the DailyReflectionCard. The right third presents the ProgressRing, Gentle Reminders, and a Deep Work call-to-action shortcut, all of which are hidden in Simplify Mode.

4.2.1 Conversational Reset

The Conversational Reset is the application's most distinctive feature. It renders an input field with the placeholder "I'm feeling a bit overwhelmed..." and a send button. On submission, the detectSentiment function classifies the input text by checking for membership in a predefined set of low-energy keywords (including 'overwhelmed', 'tired', 'anxious', 'burnout') and high-energy keywords (including 'ready', 'excited', 'motivated'). The sentiment classification is then passed to generateContextSuggestion alongside live data from the task, habit, and deep work stores.

The suggestion generator applies a decision tree over nine possible combinations of sentiment and contextual state, producing a personalised message and a contextually appropriate call-to-action link. For example, a low-energy user with pending tasks receives the message: “That’s okay. You don’t have to do everything. How about just one thing — [smallest task title]?”. A high-energy user with no focus sessions receives a prompt to begin a deep work session. The user is also presented with a “Not Yet” option that dismisses the suggestion entirely without negative reinforcement.

4.2.2 ProgressRing

The ProgressRing component renders an SVG-based circular progress indicator showing the ratio of tasks completed today to total tasks. It is implemented using an SVG circle with a stroke-dashoffset animation driven by Framer Motion, producing a smooth fill transition as tasks are completed. The component’s messaging is calibrated to be celebratory at all completion levels, never displaying zero-progress framing.

4.3 Task Management Module

The Tasks module is accessed via the /tasks route and presents the complete task list with a CaptureBar for rapid task entry at the top. Active tasks are displayed as TaskCard components with a task title, a timer duration indicator, and a completion toggle. Completed tasks are shown in a visually de-emphasised section at the bottom with strikethrough formatting.

Tasks are created through the CaptureBar with a title and an associated timer duration (in minutes), which links each task to the Deep Work module. The taskStore records a completedAt timestamp on task completion, enabling the DashboardContent to correctly filter tasks completed on the current calendar day.

The AnimatePresence component from Framer Motion wraps the active task list, providing smooth entrance and exit animations as tasks are added or completed. The layout prop on individual task motion containers ensures that sibling tasks animate smoothly into their new positions when a task is removed.

4.4 Deep Work Timer Module

The Deep Work module is the application’s most visually prominent component. Its centrepiece is a 240×240 pixel circular SVG timer with a 12-pixel stroke width. The timer arc is rendered as an animated circle with a stroke-dashoffset that decreases linearly as time elapses, driven by a custom useTimer hook that exposes formattedTime and progressPercent values.

The timer gradient transitions from blue (#1D4ED8) to cyan (#22D3EE) during normal operation and shifts to a warm teal gradient when fewer than five minutes remain, providing a visual “warning” that is calming rather than alarming. The center of the ring displays the remaining time in a monospaced font with a gradient text fill.

Session lifecycle is managed by the `deepWorkStore`. Calling `startSession` creates a new session entry with the current timestamp. `pauseSession` and `resumeSession` toggle the `isPaused` flag. `stopSession` finalises the session entry with its duration and adds it to the `sessionHistory` array. On session completion, a `showCompletion` overlay is displayed with a celebratory emoji, the session duration, and options to take a break or immediately begin another session.

The `NoiseSelector` component presents a set of ambient sound options to accompany focus sessions. The duration selector allows users to choose from 15, 25, 45, or 60-minute intervals, with the active duration highlighted using the gradient background class.

4.5 Habit Tracker Module

The Habits module implements a comprehensive habit management system. Each habit is defined by a name, icon, colour, frequency (daily, 3x weekly, 5x weekly, or custom), weekly target count, and an optional description. Habits are rendered as `HabitCard` components with an icon pill, progress bar, weekly day-dot indicators (`DayDots`), and a single-tap logging button.

4.5.1 Habit Strength Scoring

The `habitStore`’s `getStrengthScore` function computes a weekly consistency percentage by comparing the number of log entries within the current week against the sum of all habits’ weekly targets. This composite score is displayed in the `HabitStrengthCard` and used to drive the dashboard’s adaptive messaging. A score above 80% triggers an encouraging “You’ve nurtured your habits with X% consistency” message, while a score below 50% surfaces a compassionate acknowledgement of a lighter week.

4.5.2 HabitHeatmap

The `HabitHeatmap` renders a calendar grid of the past 12 weeks, with each day cell coloured according to whether any habit was logged on that date. The colour intensity scales with the number of habits logged, providing a visual density representation of overall habit engagement over time. This visualisation is intentionally low-pressure — it shows patterns rather than streaks, and does not highlight “missed” days with negative visual treatments.

4.5.3 HydrationRing

The HydrationRing is a dedicated habit-adjacent widget that tracks daily water intake in cups. It renders a circular SVG ring analogous to the focus timer, filling from 0 to 8 cups. Tap-to-increment interaction is provided by a central button, and the ring colour transitions as hydration progress increases. This component reflects research on the role of hydration in cognitive performance and serves as a persistent, low-friction daily check-in.

4.6 Notes Module

The Notes module implements a block-based editor inspired by Notion’s document model. Each note consists of a title, an optional subtitle, and an ordered array of NoteBlock objects. The NoteCanvas component renders the block array and manages drag-and-drop reordering via HTML5 drag events.

Table 4 describes the six block types supported by the Notes module.

Block Type	Description	Implementation
text	Standard paragraph text with auto-expanding textarea	TextBlock component with content prop
heading	Large, bold heading text	TextBlock with isHeading=true prop
checklist	Interactive checklist with add/toggle/delete item actions	ChecklistBlock with items array
quote	Italic serif text with left border accent in brand cyan	Inline textarea with border-l styling
divider	Visual separator with gradient fade	Decorative hr-equivalent with Minus icon
voice	Voice memo block with Web Speech API transcription	VoiceBlock with content onChange

The AddBlock button at the bottom of each note opens a BlockPicker that presents the available block types as buttons. On selection, a new block is appended to the note’s block array in the noteStore. Block deletion, available on hover, removes the target block from the array, with the constraint that notes must retain at least one block at all times.

Three note templates are provided: Morning Reflection, Project Note, and Brain Dump. Each template pre-populates the note with a set of guiding blocks appropriate to its purpose, reducing the cognitive friction of starting a new note from a blank canvas.

4.7 Analytics Module

The Analytics module, titled “Quiet Insights” in the application, takes a deliberate approach to data presentation that diverges from conventional analytics dashboards. Rather than foregrounding metrics, the module places all analytics behind a RevealGate — a single call-to-action that the user must actively choose to unlock. This design prevents involuntary exposure to performance data at moments when the user may not be ready to process it.

Table 5 summarises the analytics computation functions defined in `lib/analytics.ts`.

Function	Input	Output
<code>computeFocusPatterns</code>	<code>sessionHistory</code>	Array of daily focus minute totals for the current week
<code>getBestFocusDay</code>	<code>focusPatterns</code>	Day name with the highest cumulative focus minutes
<code>computeHabitRadar</code>	<code>habits, logs</code>	Array of <code>HabitRadarPoint</code> objects with name, icon, and strength (0–1)
<code>generateNarrativeInsights</code>	<code>sessionHistory, tasks, habits, logs, energyState</code>	Array of plain-language insight strings
<code>computeMoodSummary</code>	<code>energyState, taskCompletionRate, habitStrength</code>	Mood label, colour, and descriptive text
<code>getWeeklyAffirmation</code>	—	A weekly affirmation string selected by week-of-year index

4.7.1 Focus Pattern Bar Chart

The `FocusPatternBar` component renders a horizontal bar chart of daily focus minutes across the current week. Each bar is proportionally sized against the maximum daily total in the data set,

coloured with the brand gradient, and labelled with the day abbreviation and minute count. The best day is visually highlighted with a slightly taller bar and a label annotation.

4.7.2 Habit Strength Radar Chart

The HabitRadar component renders a custom SVG polygon chart with one axis per habit. The polygon shape is drawn by computing each habit’s angular position and radial extent (proportional to its weekly completion rate), then connecting the computed vertices. Concentric circular grid lines at 25%, 50%, 75%, and 100% provide reference levels. The polygon is filled with a semi-transparent brand gradient and stroked with a blue-to-cyan linear gradient, animated into view using Framer Motion’s scale from origin.

4.7.3 Narrative Insights

The NarrativeInsight component renders a list of plain-language insight strings generated by the generateNarrativeInsights function. This function applies a rule-based analysis of the user’s focus session history, task completion rate, habit logs, and reported energy state to produce two to four sentences that contextualise the user’s week in a supportive, non-comparative manner. Example outputs include observations such as “Your most productive window has been Tuesday mornings,” or “Your habit consistency improved this week despite a lower energy state.”

4.7.4 Weekly Affirmation

The WeeklyAffirmation component closes the analytics panel with a rotating weekly affirmation. The affirmation is selected deterministically by the week-of-year number, ensuring that the same affirmation is displayed throughout a given week while cycling to a new message each Monday. This feature embodies the project’s philosophy of replacing performance pressure with encouragement.

CHAPTER 5

EXPERIMENTAL RESULTS AND DISCUSSION

5.1 Feature Validation

Each of the six application modules was validated against its stated functional requirements through manual end-to-end testing. The following summarises the results.

The Dashboard module successfully renders all components under both normal and Recovery Mode conditions. The Conversational Reset correctly classifies low-energy inputs containing keywords such as ‘overwhelmed’ and ‘exhausted’, and produces appropriate task-reduction suggestions. High-energy inputs produce deep work and task-focused recommendations. The Recovery Banner is correctly displayed when the daysSince value exceeds the recovery threshold.

The Task Management module correctly adds, completes, and deletes tasks. The completedAt timestamp is correctly set on completion, enabling accurate same-day filtering in the Dashboard. The animated list transitions function correctly under rapid add/complete sequences.

The Deep Work Timer correctly initialises, counts down, pauses, resumes, and terminates sessions. The SVG arc animation runs smoothly at 60fps. The warm-colour transition at the 5-minute mark has been verified visually. Session history entries are correctly appended on stop and used by the Analytics module.

The Habit Tracker correctly logs daily entries, computes weekly strength scores, and renders the HabitHeatmap with correct day-alignment. The HydrationRing correctly increments and resets. The AddHabitModal validates inputs and prevents duplicate habit creation.

The Notes module correctly creates notes from blank and template starting points. Block addition, editing, deletion, and reordering via drag-and-drop all function correctly. The voice block correctly interfaces with the Web Speech API where browser support is available.

The Analytics module correctly computes and renders the Focus Pattern Bar, Habit Radar, Mood Summary, Narrative Insights, and Weekly Affirmation. The RevealGate correctly gates all content until the user chooses to reveal it. In Recovery Mode, the analytics page correctly replaces the full panel with the compassionate hide message.

5.2 UI/UX Evaluation

FocusFlow’s user interface was evaluated against established usability heuristics, with particular attention to Nielsen’s Ten Usability Heuristics and the WCAG 2.1 AA guidelines.

Consistency and Standards: The application maintains consistent use of the brand colour palette, typography scale, border radius values, and shadow depth across all modules. Navigation labels are unambiguous and the active route is clearly indicated in the sidebar through a gradient background highlight and a cyan indicator dot.

Error Prevention: The task capture bar does not permit empty task submission. The habit deletion action is gated behind a hover-revealed delete button, reducing accidental deletions. The ‘Not Yet’ option in the Conversational Reset prevents users from feeling obligated to respond to a suggestion they are not ready to act on.

Aesthetic and Minimalist Design: The interface successfully implements a minimalist visual language with a restrained colour palette, generous white space, and card-based content organisation. The use of glass morphism (‘glass-card’) creates visual depth without clutter. Recovery Mode further reduces visual complexity by collapsing the sidebar and limiting content density.

Flexibility and Efficiency of Use: Expert users can navigate between modules via keyboard (Tab + Enter for link activation). The sidebar dock-wave animation on hover provides a macOS-inspired interaction that rewards mouse-precise navigation. The capture bar in the Tasks module supports keyboard-only task creation.

5.3 Performance Considerations

FocusFlow’s client-side architecture with Zustand state management and Framer Motion animations was evaluated for runtime performance. The following observations were made.

The use of the `useMemo` hook in `AnalyticsContent` for all five analytics computations (`computeFocusPatterns`, `getBestFocusDay`, `computeHabitRadar`, `generateNarrativeInsights`, `computeMoodSummary`) ensures that expensive computations are not re-executed on unrelated state changes. Dependency arrays are specified precisely to minimise unnecessary recomputation.

The `AnimatePresence` and layout animation patterns in Framer Motion use the FLIP technique (First, Last, Invert, Play) internally, which produces smooth 60fps animations by computing transforms rather than animating layout properties. This avoids layout thrashing in the task list and note block editor.

The SVG-based timer and radar chart computations are performed in the render function but are lightweight enough (fewer than 20 trigonometric operations per render) to not require memoisation. The timer’s `stroke-dashoffset` is animated via Framer Motion’s `animate` prop with a 0.5-second linear transition, which is handled entirely on the compositor thread.

Zustand’s `persist` middleware serialises to `localStorage` on every state mutation. For the current data volumes (tens of tasks, habits, and notes), this is instantaneous. Large-scale usage with

hundreds of notes or thousands of habit logs may require selective persistence or pagination strategies in a future version.

5.4 Cross-Device Responsiveness

FocusFlow's responsive behaviour was validated across the following device categories: desktop (1440px wide), laptop (1280px), tablet (768px), and mobile (375px).

On desktop, the three-column dashboard grid, the notes sidebar-editor split, and the full sidebar all render correctly. On laptop, the layouts adapt gracefully with minor spacing adjustments. On tablet, the notes sidebar hides and mobile-style note selection is activated. On mobile, the desktop sidebar is hidden entirely and the bottom navigation bar is displayed. All page layouts use a single-column configuration on mobile.

The pb-24 md:pb-6 padding pattern on all Content components accounts for the height of the bottom navigation bar on mobile, ensuring content is not obscured by the fixed navigation element. The fixed FAB (Floating Action Button) in the Habits module uses z-[60] to render above the bottom navigation.

All interactive touch targets meet or exceed the 44×44 pixel minimum recommended by Apple's Human Interface Guidelines and the WCAG 2.1 AA criterion for pointer input, with all primary action buttons sized at a minimum of 40×40 pixels.

CHAPTER 6

CONCLUSIONS AND FUTURE DIRECTIONS

6.1 Conclusions

FocusFlow successfully demonstrates that it is possible to build a modern, production-quality productivity application that prioritises psychological wellbeing without sacrificing functional depth. The project delivered six fully operational modules — Dashboard, Tasks, Deep Work, Habits, Notes, and Analytics — within a cohesive design system, all implemented using industry-standard technologies (Next.js 14, React 18, TypeScript, Zustand, Framer Motion, Tailwind CSS).

The project’s primary technical contributions are: the Recovery Mode architecture, which demonstrates how a client-side application can adapt its interface complexity in response to inferred user state without requiring server-side computation; the Conversational Reset widget, which introduces rule-based affective computing into a productivity dashboard; the SVG-based circular timer with gradient transitions; and the block-based note editor with drag-and-drop reordering.

The project also makes a design-philosophy contribution by demonstrating, in working code, a set of EQ-first design principles — the avoidance of red feedback colours, the preference for weekly targets over daily streaks, the RevealGate pattern for analytics, and the ‘Not Yet’ interaction pattern — that collectively reduce the anxiety and shame often associated with productivity software.

All primary objectives stated in Section 1.3 were fully achieved. The application is locally deployable and functions correctly across desktop and mobile viewports.

6.2 Limitations

The following limitations are acknowledged in the current implementation.

- **Single-User, Client-Only Architecture:** The absence of a backend server and authentication system restricts FocusFlow to single-device, single-user use. Data stored in `localStorage` is not portable across devices or browsers.
- **Rule-Based Sentiment Analysis:** The Conversational Reset’s keyword-based sentiment engine is brittle and does not handle nuanced, context-dependent language, negation (e.g., “I’m not tired”), or mixed sentiment.
- **No Offline PWA Support:** The application requires a network connection for the initial load and does not register a service worker for offline functionality.

- **Limited Accessibility Audit:** While basic keyboard navigation and focus management are implemented, a comprehensive WCAG 2.1 AA audit has not been conducted. Screen reader compatibility for the SVG charts requires additional ARIA labelling.
- **Local State Persistence Limits:** Zustand persist serialises the full store on every mutation, which may become a performance concern with large datasets.

6.3 Future Work

The following directions are identified for future development phases.

6.3.1 Backend Integration and Multi-Device Sync

The most impactful near-term extension is the introduction of a backend API (REST or GraphQL) backed by a relational database (PostgreSQL or Supabase) to replace localStorage persistence. This would enable multi-device synchronisation, user authentication, and data export functionality. A Next.js API routes layer within the existing monorepo would facilitate this transition with minimal architectural disruption.

6.3.2 NLP-Enhanced Conversational Reset

Replacing the keyword-based sentiment engine with a small, locally-hosted language model (such as a quantised Llama or Mistral variant via the Transformers.js library) would substantially improve the emotional accuracy of the Conversational Reset. Edge inference via the Vercel Edge Runtime or WebAssembly-based inference could deliver this capability without the latency of a remote API call.

6.3.3 Progressive Web Application Packaging

Registering a service worker, defining a web app manifest, and implementing background sync would allow FocusFlow to be installed as a PWA on desktop and mobile platforms, providing offline functionality and a native-app-like installation experience.

6.3.4 AI-Powered Weekly Review

A weekly review feature that uses a language model to synthesise the user's focus session history, task completion patterns, habit logs, and energy state reflections into a personalised narrative review would extend the project's EQ-first philosophy into the domain of reflective practice.

6.3.5 Accessibility Hardening

A full WCAG 2.1 AA compliance audit, with particular attention to SVG chart ARIA labelling, colour contrast ratios in dark mode, and screen reader traversal of the block-based note editor, would be undertaken in a dedicated accessibility sprint.

6.3.6 Notification and Calendar Integration

Integration with the Web Notifications API for gentle, time-based habit reminders, and with the Google Calendar API or CalDAV for task deadline synchronisation, would increase FocusFlow's practical utility as a daily productivity tool.

REFERENCES

- [1] Cirillo, F. (2006). The Pomodoro Technique. FC Garage GmbH.
- [2] Clear, J. (2018). Atomic Habits: An Easy and Proven Way to Build Good Habits and Break Bad Ones. Avery.
- [3] Doyle, N. (2020). Neurodiversity at Work: A Biopsychosocial Model and the Impact on Working Adults. *British Medical Bulletin*, 135(1), 108–125.
- [4] Goleman, D. (1995). Emotional Intelligence: Why It Can Matter More Than IQ. Bantam Books.
- [5] Lally, P., van Jaarsveld, C. H. M., Potts, H. W. W., & Wardle, J. (2010). How Are Habits Formed: Modelling Habit Formation in the Real World. *European Journal of Social Psychology*, 40(6), 998–1009.
- [6] Newport, C. (2016). Deep Work: Rules for Focused Success in a Distracted World. Grand Central Publishing.
- [7] Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann Publishers.
- [8] Picard, R. W. (1997). Affective Computing. MIT Press.
- [9] Salovey, P., & Mayer, J. D. (1990). Emotional Intelligence. *Imagination, Cognition and Personality*, 9(3), 185–211.
- [10] W3C. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. World Wide Web Consortium. Retrieved from <https://www.w3.org/TR/WCAG21/>
- [11] Vercel Inc. (2024). Next.js Documentation — App Router. Retrieved from <https://nextjs.org/docs>
- [12] PMndrs. (2024). Zustand State Management Documentation. Retrieved from <https://github.com/pmndrs/zustand>
- [13] Framer. (2024). Framer Motion API Documentation. Retrieved from <https://www.framer.com/motion/>
- [14] Tailwind Labs. (2024). Tailwind CSS Documentation. Retrieved from <https://tailwindcss.com/docs>
- [15] Lucide Contributors. (2024). Lucide Icon Library. Retrieved from <https://lucide.dev>